Elective Course
Course Code: CS4103
Autumn 2026-27

Lecture #11

Artificial Intelligence for Data Science

Module-2: PROBLEM SOLVING BY SEARCH

Introduction to Informed Search [Part-III]

(Consistent Heuristics, More on Heuristics & Local Search: Introduction)

Course Instructor:

Dr. Monidipa Das

Assistant Professor

Department of Computational and Data Sciences

Indian Institute of Science Education and Research Kolkata, India 741246

Consistent Heuristics

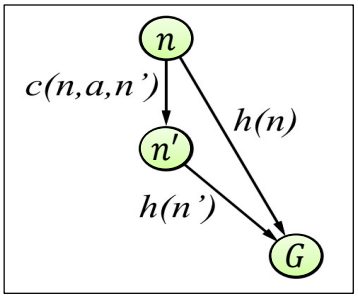
2

- **A heuristic is consistent if**

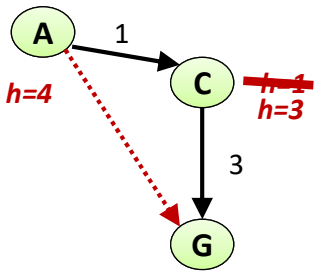
$$h(n) \leq c(n,a,n') + h(n')$$

- If h is consistent, we have

$$f(n') = g(n') + h(n')$$
$$= g(n) + c(n,a,n') + h(n')$$
$$\geq g(n) + h(n) = f(n)$$



i.e., $f(n)$ is nondecreasing along any path



Consequences of consistency:

- The *f* value along a path never decreases
- A* graph search is optimal

Optimality of A*



- **Tree search:**
  - A* is optimal if heuristic is admissible
- **Graph search:**
  - A* optimal if heuristic is consistent
- Consistency implies admissibility

Homework:
Prove that if a heuristic is consistent, it must be admissible.

Properties of A*

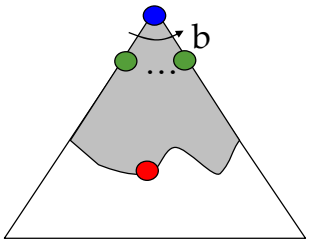


- **Complete?**
  - Yes, unless there are infinitely many nodes n with $f(n) \leq f(G)$
- **Time complexity?**
  - $O(b^{\epsilon d})$
  - where $\epsilon = |h(n_0) - h^*(n_0)| / h^*$
    n_0 = start state
    h^* = actual cost to goal state
- **Space complexity?**
  - $O(b^m)$, as in Greedy Best-First — may end up with all nodes in memory
- **Optimal?**
  - Yes, if h is admissible (Tree Search), consistent (Graph Search)

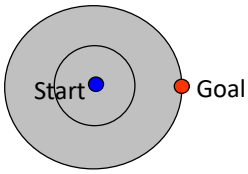
UCS vs. A*



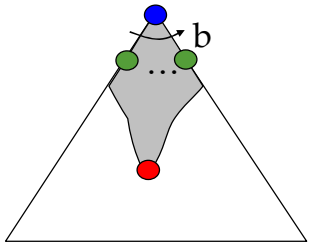
Uniform-Cost



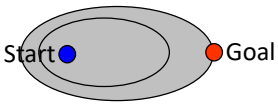
Uniform-cost expands equally in all "directions"



A*



A* expands mainly toward the goal



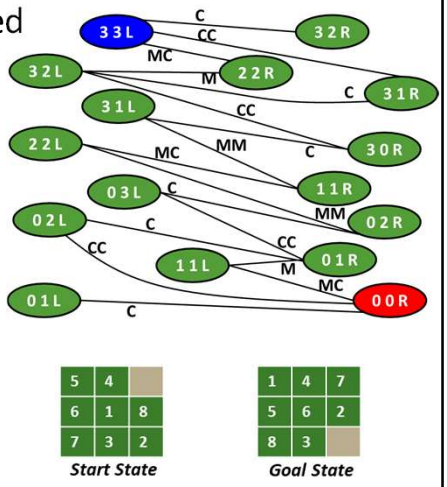


More on Heuristics

Heuristics



- **All domain knowledge** used in the search is encoded in the heuristic function $h()$.
- **Examples:**
  - Missionaries and Cannibals: Number of people on starting river bank
  - 8-Puzzle: Number of tiles out of place
  - 8-Puzzle: Sum of distances of each tile from its goal position



Heuristics [contd.]

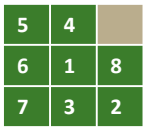


- In general:
  - $h(n) \geq 0$ for all nodes n
  - $h(n) = 0$ implies that the node n is a goal node
  - $h(n) = \infty$ implies that n is a dead-end that can never lead to a goal

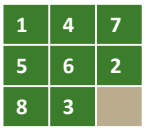
Examples of Admissible Heuristics



- **8-puzzle problem:**
  - $h_1(n)$ = number of tiles in the wrong position at state n
  - $h_2(n)$ = sum of the Manhattan distances of each tile from its goal position



Start State



Goal State

  - $h_1(Start)$ = 6
  - $h_2(Start)$= 2+1+0+0+1+1+4+3 = 12

Effectiveness of Heuristic Functions



- Let
  - h be a heuristic function for A*
  - N be the total number of nodes expanded by one A* search with h
  - d is the depth of the found solution

- The effective branching Factor (EBF) of h is the value b* that solves the equation

$$x^d + x^{d-1} + \cdots + x^2 + x + 1 - N = 0$$

(the branching factor of a uniform tree with N nodes and depth d)

A heuristics h for A* is effective in practice if its average EBF is close to 1

Dominance



- A heuristic function h_2 dominates a heuristic function h_1 for a problem P if $h_2(n) \geq h_1(n)$ for all nodes n in P's space
- **Ex.:** the 8-puzzle
  - h_2 = total Manhattan distance dominates
  - h_1 = number of misplaced tiles
- With A*, if h_2 is admissible and dominates h_1, then it is always better for search: A* will never expand more nodes with h_2 than with h_1
- **Note:** If h_2 dominates h_1, then $EFB(h_2) \leq EFB(h_1)$
- **What if neither of h_1, h_2 dominates the other?**
  - If both h_1, h_2 are admissible, use $h(n) = \max(h_1(n), h_2(n))$

Devising Heuristic Functions



- How do we devise good heuristic functions for a given problem?
- Typically, that depends on the problem domain
- However, there are some general techniques that work reasonably well across several domains
  - **Relaxation of formally described problems**
  - Pattern databases
  - Learning

Generating Admissible Heuristics



- A **relaxed problem** is a version of a search problem with less restrictions on the applicability of the next-state operators
- **Example: n-puzzle**
  - original: "A tile can move from position p to position q if p is adjacent to q and q is empty"
  - relaxed-1: "A tile can move from p to q if p is adjacent to q "
  - relaxed-2: "A tile can move from p to q if q is empty"
  - relaxed-3: "A tile can move from p to q "
- The exact solution cost of a relaxed problem is often a good (admissible) heuristics for the original problem
- **Key point:** the optimal solution cost of the relaxed problem is no greater than the optimal solution cost of the original problem



Beyond Classical Search

Local Search: Hill Climbing

Local Search

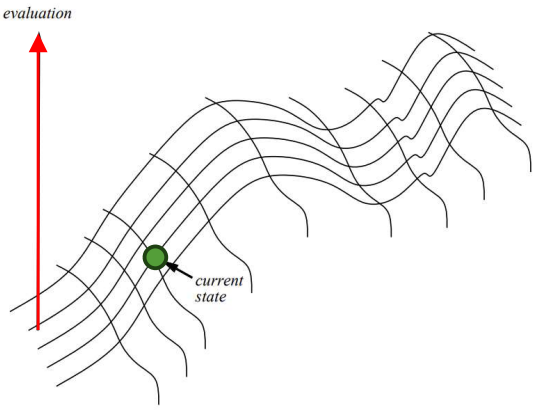


- In many optimization problems the goal state itself is the solution
- The state space is a set of complete configurations
- Search is about finding the *optimal* configuration or just a *feasible* configuration
- In such cases, one can use *iterative improvement*, or *local search*, methods
- An evaluation, or *objective*, function h must be available that measures the *quality* of each state
- **Main Idea:** Start with a random initial configuration and make small, local changes to it that improve its quality

Local Search: The Landscape Metaphor



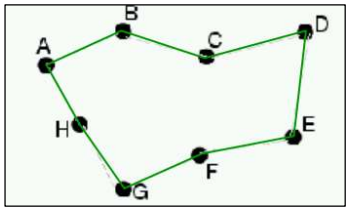
- Ideally, the evaluation function h should be *monotonic*: the closer a state to an optimal goal state the better its h-value.
- Each state can be seen as a point on a surface.
- The search consists in moving on the surface, looking for its highest peaks (or, lowest valleys): the optimal solutions.



Local Search Example: TSP

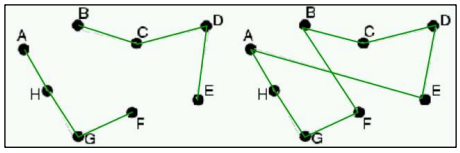


- **TSP:** Travelling Salesperson Problem
- h = length of the tour
- **Strategy:** Start with any complete tour, perform pairwise exchanges

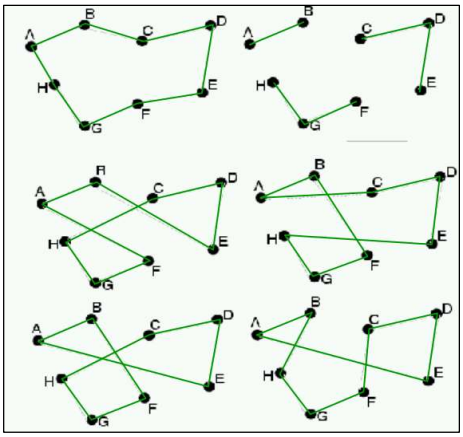


- Variants of this approach get within 1% of optimal very quickly with thousands of cities

2-Change Example



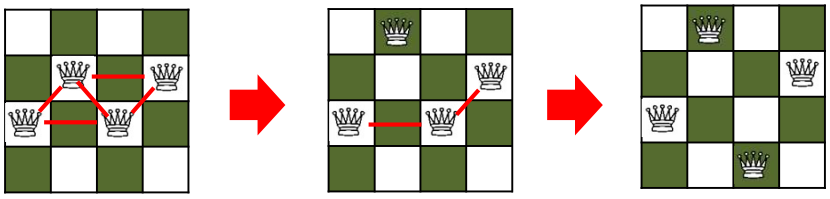
3-Change Example



Local Search Example: n-queens



- Put n queens on an $n \times n$ board with no two queens on the same row, column, or diagonal
- h = number of conflicts
- **Strategy:** Move a queen to another square in the same column



$h = 5$ $h = 2$ $h = 0$

- Almost always solves n-queens problems almost instantaneously for very large n, e.g., $n = 10^6$

Local Search: Boolean Satisfiability or SAT Problem



- **SAT:** determining if a Boolean formula is satisfiable or unsatisfiable.
  - **Satisfiable :** If the Boolean variables can be assigned values such that the formula turns out to be TRUE
- $\boldsymbol{h}$ = number of satisfied clauses
- **Current state:** (A=T, B=F, C=T, D=T, E=T)
- **Strategy:** flip the assignment of one variable

(A=**F**, B=F, C=T, D=T, E=T)
(A=T, B=**T**, C=T, D=T, E=T)
(A=T, B=F, C=**F**, D=T, E=T)
(A=T, B=F, C=T, D=**F**, E=T)
(A=T, B=F, C=T, D=T, E=**F**)

$A \vee \neg B \vee C$
$\neg A \vee C \vee D$
$B \vee D \vee \neg E$
$\neg C \vee \neg D \vee \neg E$
$\neg A \vee \neg C \vee E$

Next Lecture (on Local Search Techniques): Hill Climbing and Simulated Annealing



Questions?